

Bulgarian Diploma Thesis

Used Cars Price Predictor
with Machine Learning

Aleksandar Ivanov, ID#200231727

Student:  , Date: 05.12.2025
signature

Supervisor: _____ , Date: 05.12.2025
signature

Department of Computer Science AUBG

Blagoevgrad 2025

Declaration for Senior Project and Diploma Thesis

Title: Used Cars Price Prediction

Author: Aleksandar Ivanov

Car valuation has usually relies on manual analysis of the market or static pricing models. These methods often produce inconsistent or outdated estimates, because they fail to find the connection between the different vehicles and their features. The goal of this project is to address these limitations by developing a full-stack web application. The application is going to be using the following machine learning algorithms for predicting the prices of used cars - Linear Regression, Ridge Regression, Random Forest, and Gradient Boosting. The core of the project is the backend and it is implemented in ASP.NET Core. The frontend is developed with React and TypeScript. The backend handles model training, evaluation, and serialization. The frontend provides an intuitive interface for predictions, trend visualization across time ranges, and side-by-side model comparisons via charts. In the data preprocessing pipeline there are data quality checks that verify for missing values and outliers. Also there is feature encoding and normalization. The models are comapred and evaluated using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and coefficient of determination (R^2). Ridge regression combined with ensemble residual learners (Random Forest and Gradient Boosting) gave the most accurate performance.

Declaration of authorship:

“The Senior Project/Bulgarian Diploma Thesis presented here is the work of the author solely, without any external help, under the supervision of Narasimha Rao Vajjhala. All sources, used in development, are cited in the text and in the Reference section.”



Author: _____

Table of Contents

1. Introduction	5
1.1 Motivation.....	5
1.2 Project Overview	6
2. Specification of the Software Requirements and their Analysis.....	7
2.1 Functional Requirements	7
2.2. Non-functional Requirements.....	9
2.3. Requirements Analysis.....	9
2.3.1. Analysis of Functional Requiriements	9
2.3.2. Analysis of Non-functional Requirements.....	19
3. Design of Software Solution	20
3.1. Main algorithms	20
3.1.1. Linear Regression.....	20
3.1.2. Ridge Regression.....	21
3.1.3. Random Forest Regression	21
3.1.4. Gradient Boosting Regression	22
3.2. Algorithm Complexity.....	24
3.2.1. Linear Regression Complexity	24
3.2.2. Ridge Regression Complexity.....	25
3.2.3. Random Forest Complexity.....	25
3.2.4. Gradient Boosting Complexity.....	26
3.3. Software Architecture	26
3.3.1. Model(s)	28
3.3.2. View(s).....	30
3.3.3. Controller(s).....	34
3.4. Primary Vehicle Dataset and Feature Selection.....	35
3.5. Custom Dataset Requirements	36
4.Implementation	36
4.1. Development Environment and Programming Languages	36
4.2. External Libraries and Frameworks.....	37

4.3. Installation Requirements and Project Setup	39
4.4. Code Fragments	41
4.4.1. Ridge Regression	41
4.4.2. DecisionTreeRegressor (Base Component)	42
4.4.3. Ensemble Algorithms: Random Forest and Gradient Boosting	43
5. Testing	44
5.1 Evaluation Metrics and Procedures	45
5.1.1. Coefficient of Determination (R^2)	45
5.1.2. Root Mean Squared Error (RMSE)	46
5.1.3. Mean Absolute Error (MAE)	47
5.1.4. Training Time and Efficiency	48
5.2. Functional Requirements Testing - Backend	48
5.2.1. Prediction Algorithms and Model Loading - Passed	49
5.2.2. Data Preprocessing and Encoding - Passed	49
5.2.3. API and Controller Reliability - Passed	50
5.2.4. Minimum Prediction Accuracy - Passed	51
5.3. Functional Requirements Testing - Frontend	51
5.3.1. UI Components & State Handling - Passed	51
5.3.2. Form Validation & User Flows - Passed	51
5.3.3. Data Transformation & View Consistency - Passed	52
5.3.4. Integration with Backend Contracts (Mocked) - Passed	52
5.3.5. Coverage Summary	52
6. Results and Conclusion	53
6.1 Experimental Setup	53
6.2 Algorithm Performance (Single Prediction)	53
6.3 Price Prediction Over Time (Range Prediction)	54
6.4 Comparison Views	57
6.5 Future Developments	58
6.6 Conclusion	59
7. References	60

1. Introduction

1.1 Motivation

One of the fastest-moving parts of the global automotive industry is the used car market. Each year, millions of cars are sold and bought in the second-hand market and their prices are influenced by a lot of factors like brand, model, mileage, year of production, engine, transmission and fuel type (gasoline, diesel, hybrid, electric). For buyers who are not aware of the details in the pricing of vehicles it can lead to overpaying. On the other side, for sellers, inaccurate research on how they should price their vehicle may lead to losses or overpricing it which will cause delays in selling.

Usually, the valuation of vehicles is handled manually or by online platforms that rely on historical average (Amik et al.). However, these methods are not the most optimal as of today. Expert-based evaluations can be subjective, time-consuming and difficult to scale across large markets (Alnajim et al.). Using online platforms, while they are automated way to predict prices, they often fail to make the right prediction (Pal et al.; Amik et al.). These inaccuracies can cause a huge financial damage to the sellers because the market is very competitive.

Machine learning (ML) gives us a new and improved way to predict prices for vehicles. By using large datasets from previously sold vehicles, ML can find patterns. ML can find relations in data that is non-linear like such as brand, model, mileage, year of production which will lead to calculations which are more precise. Using supervised algorithms such as linear regression, gradient boosting, random forest accuracy is significantly improved compared to manual and rule-based techniques (Amik et al.; Alnajim et al.; Pal et al.).

The motivation behind this project is to make a working ML-powered web application that can give correct predictions to prices on used vehicles given their key attributes. Both buyers and sellers can benefit from such system because it will introduce a lot more transparency and reducing the risk of overvaluation or undervaluation. It will make dealerships more competitive as they are now going to be able to adapt to the price changes much quicker. This project can also be of use of academic purpose as it shows how ML techniques can be applied to help a real-world business problem. In this way, the project how data-centric approaches can be used to modernize an industry which relies on subjective judgement.

1.2 Project Overview

The project is a data-driven web application for predicting used-car prices. The backend is implemented in ASP.NET Core using MVC controllers that expose RESTful Web API endpoints; the frontend is a React + TypeScript single-page application that calls these endpoints. The system trains and serves four algorithms: Linear Regression, Ridge Regression, Random Forest, and Gradient Boosting.

Preprocessing is handled by dedicated components: a `FeatureScaler` for numeric inputs and a `LabelScaler` for target prices. The `LabelScaler` inverts predictions back to the original currency scale. These components are reusable across training and inference and are serialized with the trained models so the web app uses the exact same pipeline used during training.

In its core the application integrates several machine learning algorithms, including Linear Regression, Gradient Boosting and Random Forest. These algorithms will be trained on publicly available used-cars datasets, which contain the following information: brand, model, year of production, transmission, engine type and fuel type. The system also integrates preprocessing components like a `FeatureScaler` and `LabelScaler`, which were implemented specifically for this project. The *FeatureScaler*

is responsible for normalizing the numeric input features (e.g., mileage, production year) into a standardized range before training. The *LabelScaler* performs the inverse operation by rescaling the model's predicted values back to the original monetary scale. These components form part of the backend preprocessing pipeline and are integrated into the application's overall architecture as independent, reusable modules.

The project has two main goals. On the practical side, it is a tool which can be used by dealerships/sellers and buyers to make the process of buying a used car more transparent and aligned with the current market. And on the academical side it can be used as a case-study for what machine learning is most suitable by enabling direct comparison between the models in terms of their accuracy and computational cost and time cost.

In addition, the project aims to provide the end user the full pipeline not only a command line interface app by implementing user-friendly web experience. Which will demonstrate how each of the algorithms perform under the same conditions and the results will be shown to the end-user in a way that a non-expert can use them. By combining everything mentioned the project will provide a robust solution for used car predictions.

2. Specification of the Software Requirements and their Analysis

2.1 Functional Requirements

- **Algorithms:** The system must train and serve price predictions using Linear Regression, Ridge Regression, Random Forest, and Gradient Boosting. All models are trained offline using the CLI trainer and exported as JSON bundles.
- **Single-car prediction:** The user selects manufacturer, model, year of production, mileage, fuel type, transmission, and target year. The system returns a price prediction from each algorithm, including evaluation metrics and model metadata.
- **Single-car range prediction:** For a selected car, the system must provide a year-by-year projection over a chosen range (e.g. 2025–2030). Ridge Regression is used to generate the trend curve because Random Forest and Gradient Boosting do not reliably extrapolate outside the training year.
- **Two-car prediction:** Users can configure two different vehicles and request single-year predictions. The backend returns predictions from all algorithms for both vehicles, enabling direct side-by-side comparison.
- **Two-car range comparison:** The application must support range-based comparison between two vehicles, producing two depreciation curves over the selected horizon.
- **Manufacturer and model catalog:** The backend inspects available trained bundles and returns a list of manufacturers and a list of models for each manufacturer. Model names must be normalized consistently using canonical bundle identifiers.
- **Model details endpoint:** For each model bundle, the system must provide fuel types, transmission types, min/max production years, and the anchor target year used during training.
- **Categorical validation:** All prediction requests must validate fuel and transmission values against a static allow-list and the bundle's own categorical vocabularies. Invalid requests must return a structured 400 error.
- **Training pipeline (CLI):** The CLI trainer must preprocess the dataset, encode categorical features, compute engineered features (age and year offset), scale numeric features, and train all four algorithms. It must enforce quality gates, skipping models with insufficient data or poor performance metrics.

- Evaluation: During training, the system must compute MAE, RMSE, and R^2 for each algorithm and store these metrics in the output bundle. The backend must return these metrics in all prediction responses.
- Web interface: The frontend should be implemented as a React single-page application. It will communicate with the ASP.NET Core backend via JSON REST APIs. The UI must display predictions, algorithm metrics, comparison tables, and range-based charts for both single-car and two-car scenarios.

2.2. Non-functional Requirements

1. The application should be configurable. It should allow selection and modification for the specific vehicle.
2. The system should achieve a level of accuracy, with at least one algorithm reaching an $R^2 \geq 0.7$ on validation performance.
3. The web interface should be intuitive and easy to use. It must present predictions and comparison charts in a clear and understandable way.
4. The web application should be easily runnable on common operating systems (Windows, macOS, Linux).

2.3. Requirements Analysis

2.3.1. Analysis of Functional Requirements

The core of the application lies in its machine learning algorithms. These were selected to provide a balance between interpretability, accuracy, and computational efficiency.

Linear Regression was included because of its simplicity. It provides an baseline against which more advanced models can be compared and also they can expand on.

However, linear regression is often insufficient for capturing the nonlinear relationships. Which is the case of this project (J. I. Kang et al.).

Ridge Regression extends linear regression by introducing regularization. It penalizes overly complex models and reduces the chances of overfitting. This makes them more stable for datasets that include correlated variables (Ting Tin Tin et al.).

Random Forest Regression was chosen due to its ability to handle nonlinear data. Research reports that Random Forest achieves strong performance in car price prediction tasks (J. I. Kang et al.; Gegic et al.).

Gradient Boosting was selected because it incrementally corrects errors of weaker learners. And it could produce the most accurate results. While it requires more computation and careful tuning, boosting methods are reported to achieve great performance in car price prediction tasks (J. I. Kang et al.).

To demonstrate the implementation requirements, the algorithms can be summarized in pseudocode:

High-level training loop:

```
1. procedure Evaluate(model, X_test, y_test, scaler)
2.   pred_scaled ← Predict(model, X_test)
3.   pred ← InverseTransform(scaler, pred_scaled)
4.   truth ← InverseTransform(scaler, y_test)
5.   return { MAE, RMSE, R2 } between pred and truth
6. end procedure
7.
8. procedure EvaluateCombined(ridge, residualModel, X_test, y_test, scaler)
9.   base ← Predict(ridge, X_test)
10.  resid ← Predict(residualModel, X_test)
11.  combined ← base + resid
12.  pred ← InverseTransform(scaler, combined)
13.  truth ← InverseTransform(scaler, y_test)
```

```

14.     return { MAE, RMSE, R2 } between pred and truth
15. end procedure

```

Data preparation and baseline models

```

1. procedure TrainSingleModel(rows, anchorYear, maxConfigs, modelName)
2.   (X_raw, y_raw, fuels, transmissions) ← Preprocess(rows, anchorYear)
3.   (X_trRaw, y_trRaw, X_teRaw, y_teRaw) ← Split(X_raw, y_raw, 0.8)
4.   featScaler ← Fit FeatureScaler on X_trRaw
5.   labelScaler ← Fit LabelScaler (log) on y_trRaw
6.   X_train ← Transform(featScaler, X_trRaw)
7.   X_test ← Transform(featScaler, X_teRaw)
8.   y_train ← Transform(labelScaler, y_trRaw)
9.   y_test ← Transform(labelScaler, y_teRaw)
10.  linear ← Fit LinearRegression on (X_train, y_train)
11.  ridge ← RidgeCV(X_train, y_train, labelScaler)

```

Residual training (RF and GB)

```

1.   (X_tr1, y_tr1, X_val1, y_val1) ← Split(X_train, y_train, 0.75)
2.   ridge_tr ← Predict(ridge, X_tr1)
3.   ridge_val ← Predict(ridge, X_val1)
4.   trainRes ← y_tr1 - ridge_tr
5.   valRes ← y_val1 - ridge_val
6.   rf ← RF_TrainResiduals(X_tr1, trainRes, X_val1, valRes, maxConfigs)
7.   gb ← GB_TrainResiduals(X_tr1, trainRes, X_val1, valRes, maxConfigs)
8.   SaveModelBundle({linear, ridge, rf, gb, featScaler, labelScaler},
9.                   meta: {anchorYear, rows count, modelName})

```

Evaluation helpers

```

1. procedure Evaluate(model, X_test, y_test, scaler)
2.   pred_scaled ← Predict(model, X_test)
3.   pred ← InverseTransform(scaler, pred_scaled)
4.   truth ← InverseTransform(scaler, y_test)
5.   return { MAE, RMSE, R2 } between pred and truth
6. end procedure
7. procedure EvaluateCombined(ridge, residualModel, X_test, y_test, scaler)
8.   combinedScaled ← Predict(ridge, X_test) + Predict(residualModel, X_test)

```

```
9.     pred <- InverseTransform(scaler, combinedScaled)
10.    truth <- InverseTransform(scaler, y_test)
11.    return { MAE, RMSE, R2 } between pred and truth
12. end procedure
```

Another essential requirement is the data preprocessing pipeline. Raw vehicle listings often contain missing values, outliers, and inconsistent formats. Without cleaning, models may learn spurious relationships or perform poorly on unseen data. To address this, the pipeline will include:

- Handling missing values (forward fill or row removal when thresholds are exceeded).
- Detecting and removing outliers using statistical measures such as the z-score.
- Encoding categorical variables (e.g., transmission, fuel type).
- Scaling numerical features for models sensitive to feature magnitude.

These preprocessing steps are consistently identified as key to improving generalization and stability in predictive modeling (Gegic et al.; Haque et al.). The process is summarized in Figure 1:

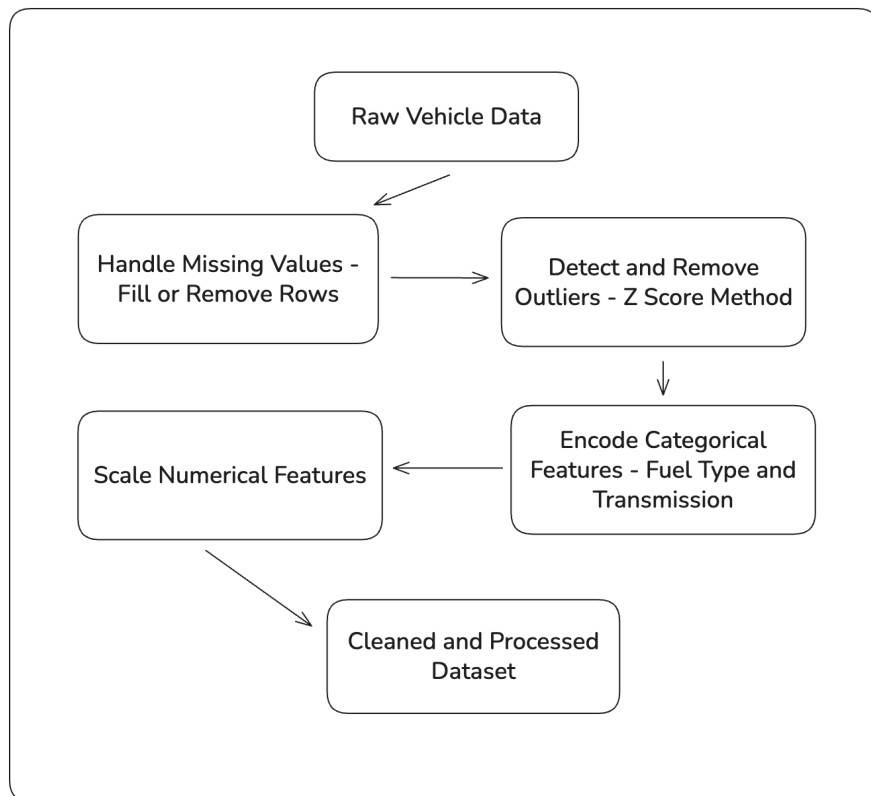


Figure 1 Preprocessing and processing activity diagrams.

A well-designed user interface is what distinguishes a good software package from an average one. A simple, intuitive, and comprehensive dashboard drastically improves user experience and accessibility. The dashboard in this application is designed to hide the complexities of the underlying machine learning algorithms, allowing the user to obtain predictions and comparisons without needing to understand the technical implementation details.

The interface follows the following structure (as seen on Figure 2):

- Main navigation menu providing access to four prediction-related views, allowing users to switch easily between modes.
- Input configuration for vehicle details such as manufacturer, model, year of production, mileage, fuel type, transmission, and target year.

- Simultaneous algorithm evaluation, where all implemented models (Linear, Ridge, Random Forest, Gradient Boosting) are executed automatically for a given input.
- Real-time results display, showing the predicted price for each algorithm without requiring additional user tuning.
- Simplified workflow, focusing on single-car predictions rather than batch or manual hyperparameter selection.
- Visual feedback, including charts and tables summarizing predicted values, trends across years, and evaluation metrics (MAE, RMSE, R^2).
- User-friendly design, emphasizing clarity, minimal configuration, and fast comparison between algorithms' outputs.

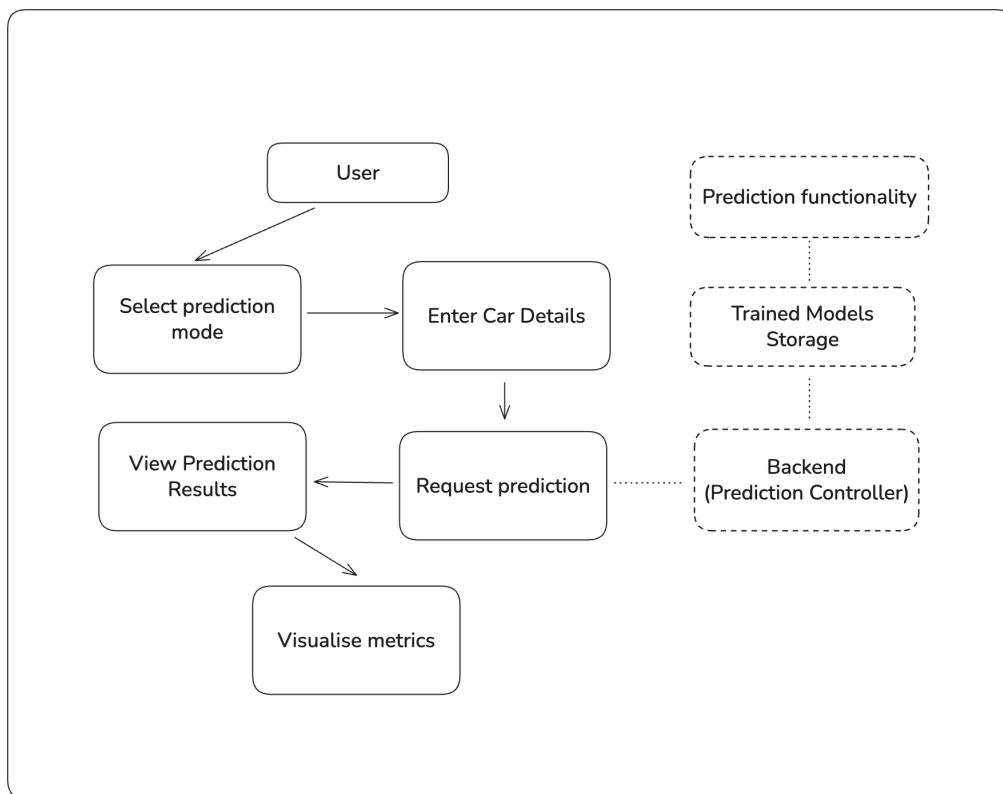


Figure 2. Use case diagram for the user dashboard.

The workflow of the system consists of - preprocessing, training, evaluation(prediction) and visualization. The application has preprocessed the data by handling missing values

and outliers. Each algorithms predictions are visualized through charts and error metrics. Finally, the system provides facilities to compare models under identical conditions. The process is visualised can be seen on Figure 3.

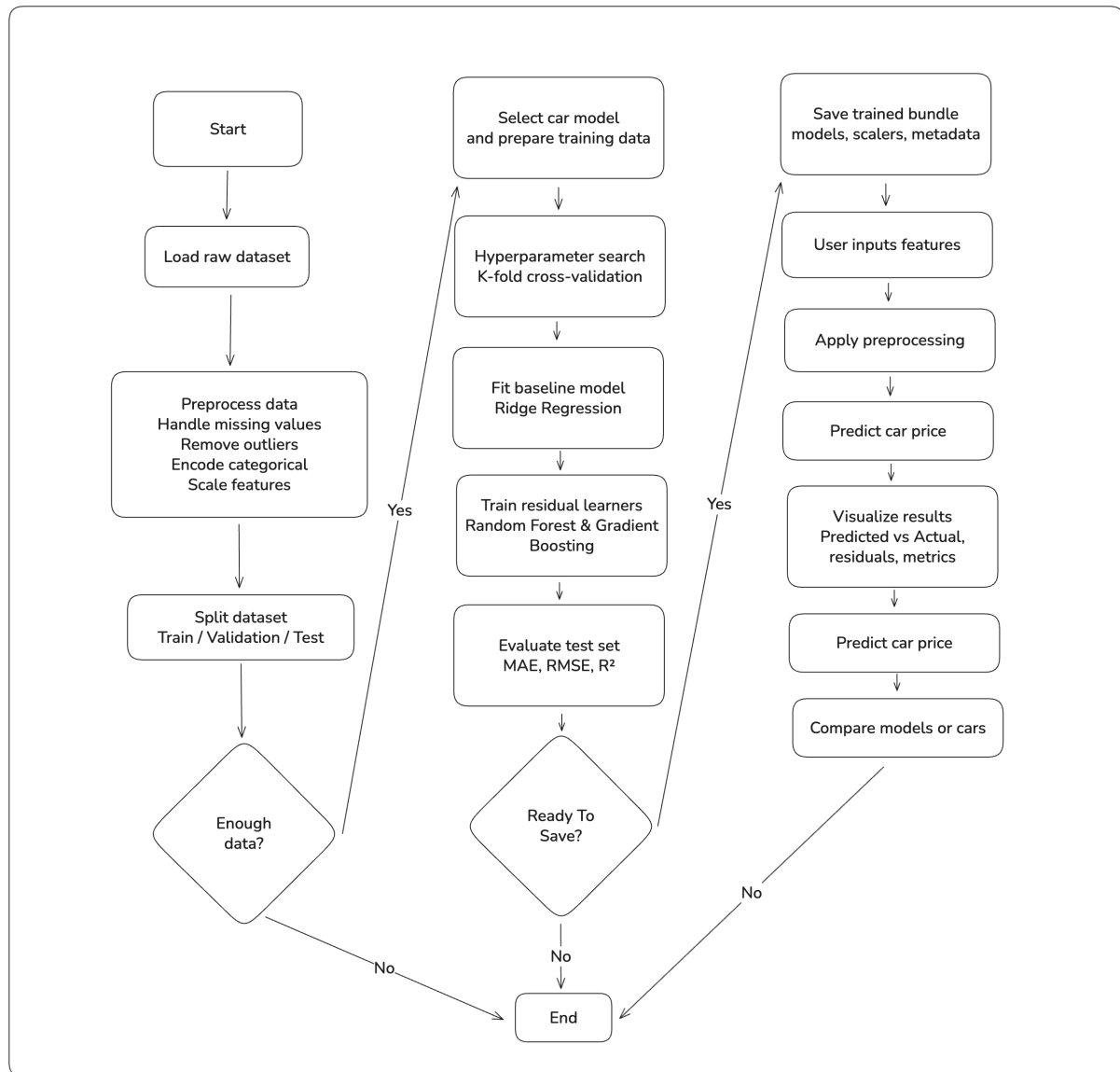


Figure 3. Activity diagram for the system workflow

Finally, the performance of the regression algorithms is evaluated using metrics. The most common in regression tasks are the coefficient of determination (R^2), the mean absolute error (MAE), and the root mean squared error (RMSE). The R^2 value reflects how predictions fit real data, with values near 1 indicating a strong fit. RMSE provides an error measure but is sensitive to outliers. And MAE is more robust to extreme

values. Together, these three metrics provide a balanced assessment of the performance of each algorithm.

Based on prior research in car price prediction, ensemble models such as Random Forest and Gradient Boosting are expected to achieve the highest R^2 values, typically above 0.8 on large datasets (J. I. Kang et al.; Gegic et al.). Linear Regression and Ridge regression generally produce lower scores, often around 0.6-0.7 (Ting Tin Tin et al.). These score results are not guaranteed because the actual performance depends on dataset quality, preprocessing, and parameter tuning.

Example Use Case Narratives

i. Name: Single Prediction

- Actor: User
- Entry Condition: The application is running and pre-trained models are loaded.
- Main Flow:
 1. Via the home screen, the user selects Single Prediction.
 2. The user selects the car's make, model, year, mileage, fuel type, and transmission.
 3. The system uses all available algorithms (Linear, Ridge, Random Forest, Gradient Boosting) to predict the price.
 4. The result screen displays numeric predictions along with performance metrics.
- Result Screen: Shows predicted prices from each algorithm, the performance metrics MAE, MSE, R^2 , and training/inference timings.
- Exit Condition: The user clicks Back button to go to the previous page and make a new request or Home to return to the main menu.

- Exceptions: If required attributes are missing, the system prompts shows the user what error they have made prompting them to fix it before trying again.

ii. Name: Single Ranged Prediction

- Actor: User
- Entry Condition: The application is running and pre-trained models are loaded.
- Main Flow:
 1. Via the home screen, the user selects Single Ranged Prediction.
 2. The user selects the car's make, model, year, mileage, fuel type, and transmission and defines a range of production years.
 3. The system uses all available algorithms (Linear, Ridge, Random Forest, Gradient Boosting) to predict the price.
 4. The result screen visualizes how price changes over time.
- Result Screen: A multi-line chart (year vs. predicted price), with one line per algorithm, and a table containing MAE, MSE, R², and timing data.
- Exit Condition: The user clicks Back button to go to the previous page and make a new request or Home to return to the main menu.
- Exceptions: If required attributes for the car are missing or incorrect, the system prompts shows the user what error they have made prompting them to fix it before trying again. Invalid year ranges (e.g., min $\geq$ max) prompt the user to correct inputs.

iii. Name: Compare Predictions (Two Cars)

- Actor: User
- Entry Condition: The application is running and pre-trained models are loaded.
- Main Flow:
 1. Via the home screen, the user selects Compare Predictions.

2. The user selects two different car configurations (make, model, and attributes) as described in the previous narratives.
 3. The system applies all algorithms to both cars and computes their predicted prices.
 4. The result screen presents the comparison.
- Result Screen: A bar chart showing predicted prices for each algorithm per car, with MAE, MSE, R^2 , and timing metrics.
 - Exit Condition: The user clicks Back button to go to the previous page and make a new request or Home to return to the main menu.
 - Exceptions: If required attributes for the car are missing or incorrect, the system prompts shows the user what error they have made prompting them to fix it before trying again.

iv. Name: Compare Ranged Predictions

- Actor: User
- Entry Condition: The application is running and pre-trained models are loaded.
- Main Flow:
 1. Via the home screen, the user selects Compare Ranged Predictions.
 2. The user selects two cars and their specifications and then chooses one algorithm for comparison.
 3. A range of years is defined.
 4. The system generates predictions for both cars over the year range using the selected algorithm.
 5. The result screen displays the trends.
- Result Screen: A two-line chart (year vs. predicted price) comparing both cars using the chosen algorithm, with corresponding MAE, MSE, R^2 , and timing information.

- Exit Condition: The user clicks Back button to go to the previous page and make a new request or Home to return to the main menu.
- Exceptions: If required attributes for the car are missing or incorrect, the system prompts shows the user what error they have made prompting them to fix it before trying again or if no algorithm or invalid year range is selected, the system displays a warning.

Notes

- Every result screen includes:
 - Predicted price(s)
 - MAE, MSE, R^2 , and timing metrics
 - Appropriate chart (line or bar) for visual comparison
- Each use case starts from Home and ends with a Back Home button.
- The system ensures consistency and reproducibility across all prediction modes.

2.3.2. Analysis of Non-functional Requirements

The scalability and configurability requirements are defined to make sure that the application is efficient and adaptable. The backend is developed using ASP.NET Core. This allows for independent training, and prediction components. This also guarantees scalability, such as adding new algorithms or modifying the data pipeline.

Configurability is achieved through the command-line interface in the form of optional arguments that control the training process. They include the anchor year, maximum number of hyperparameter search configurations, maximum dataset size, and quality-gate thresholds (minimum R^2 , maximum MAE, maximum RMSE). They allow users to

fully customize training duration, model quality requirements, and computational workload according to their specific needs.

Usability is achieved with the user interface(UI) implemented with React and Typescript. It provides four prediction modes - Single, Single Ranged, Compare, and Compare Ranged. Each mode offers user friendly UI with fast response times, visualizations with bar and line charts and performance metrics.

Finally, the system's responsiveness and maintainability are guaranteed by the use of pre-trained models and compliance to the MVC architecture, which ensures clean code and scalability.

3. Design of Software Solution

3.1. Main algorithms

3.1.1. Linear Regression

Linear Regression provides an interpretable baseline against which more advanced models can be compared to and expand on. However, linear regression is often insufficient for capturing the nonlinear relationships that exist in car price data, such as the interaction between mileage and production year (J. I. Kang et al.).

However, linear relationships are often unrealistic in the used car case. Real-world depreciation curves tend to be nonlinear for instance: mileage might reduce value steeply initially. But the marginal effect slows as the car ages. Researchers have

proposed S-curve because linear fits lead to increasing errors at extreme values (Salim and Abu).

Moreover, linear regression is vulnerable to outliers. In used car datasets, extreme prices or rare models can disproportionately skew the regression line. Which reduces robustness (Amik et al.). Also, when features like age and mileage go hand by hand, coefficient estimates can become unstable and misleading (Ting Tin Tin et al.).

3.1.2. Ridge Regression

Ridge Regression is a regularized linear model that balances interpretability and predictive performance by introducing a penalty to reduce overfitting. It applies L2 regularization. It adds a squared penalty to the magnitude of the coefficients. This lowers large weights and stabilizes estimates when they correlated. It reduces variance at the cost of a small bias.

Research has shown that Ridge Regression is a well suited algorithm for price prediction tasks and it offers a reliable balance between accuracy and stability (Zhang and Politis). It performs particularly well when most predictors contribute moderately to the result.

In used car price prediction, Ridge Regression is demonstrating greater consistency compared to linear regression. Which can be seen with the presence of multicollinearity and noisy features (Bergmann and Feuerriegel).

3.1.3. Random Forest Regression

Random Forest Regression builds a large number of decision trees, each trained on a different sample of the data. In the end it combines their outputs to generate predictions (Breiman). This reduces overfitting, which is a big issue for single decision trees. The way it works is it gets the average of the results of many trees, then Random Forest captures complex patterns and interactions between features that models like linear regression miss.

For predicting price predictions Random Forest is proven to be one of the most effective algorithms. It performs well because it can naturally model the nonlinear relationships between features such as mileage, age, brand, and engine size without requiring heavy preprocessing (Pal et al.; Bergmann and Feuerriegel). Research has shown that Random Forest models consistently outperform basic regression techniques in accuracy (Pal et al.).

An advantage for Random Forest is its ability to remain stable and reliable even when data contains noise or outliers. Since predictions are averaged across many independent trees, anomalies as mispriced listings have a limited effect on the final prediction (Mentch and Zhou). Another important feature is that the algorithm estimates feature importance. This helps to determine which variables influence the predicted price. Studies find that mileage and model year have the highest importance scores, while others like fuel type play smaller roles (Molnar).

However, Random Forest also has limitations. It requires more computational resources than simpler algorithms.. When the dataset contains weak or redundant features, its performance can also decrease slightly unless preprocessing and feature selection are applied carefully (Scornet).

3.1.4. Gradient Boosting Regression

Gradient Boosting Regression is a machine learning algorithm that builds models sequentially. Each new model focuses on correcting the errors made by the previous ones. It can be explained with an golf example. Where a person shoots, makes a mistake analyses their mistake and shoots again in an attempt to make a better shot.

It optimizes a loss function by fitting to the residuals of prior predictions. This allows it to improve performance over time (Friedman). This process enables Gradient Boosting to learn nonlinear relationships and feature interactions. This advantage is very useful in car price prediction, where variables such as mileage, age, etc.

In practice, one of the most widely used implementations of Gradient Boosting is Extreme Gradient Boosting (XGBoost). Research applying XGBoost to vehicle pricing has demonstrated high predictive accuracy when paired with proper preprocessing and parameter tuning (Qian). Similarly, Gradient Boosting is among the best performing algorithms for predicting used car prices (Bergmann and Feuerriegel).

A major strength of Gradient Boosting is the control over bias and variance through parameters such as the learning rate, tree depth, and number of estimators. Each tree in the sequence attempts to reduce the residual errors of the previous ones. This enables the model to capture data patterns that simpler algorithms often miss. However, this flexibility also makes very prone to overfitting if parameters are not carefully tuned. Multiple studies highlight the importance of hyperparameter optimization and early stopping to prevent overfitting and ensure stable results (Zheng).

Gradient Boosting is also useful for interpretability tools such as feature importance. This allow users to understand how individual features influence price predictions. When compared to Random Forest, Gradient Boosting requires more resources and a more complex training process.

3.2. Algorithm Complexity

The algorithms chosen for this senior project were chosen for their accuracy when it comes to used car prediction systems and also their efficiency which is proven by the researchers.

3.2.1. Linear Regression Complexity

Linear Regression is used in the system as the simplest baseline model. Unlike the classical formulation that relies on solving the normal equation $(X^T X)^{-1} X^T y$, this project implements Linear Regression using batch gradient descent. This approach avoids the need for matrix inversion and provides better scalability when the number of features grows.

For a dataset with n samples and m features, one gradient-descent epoch requires computing predictions and gradients for all samples, which has a cost of:

$$O(nm)$$

If training runs for E epochs, the total time complexity becomes:

$$O(E \cdot n \cdot m)$$

This makes gradient descent computationally efficient for moderate feature sizes and numerically stable compared to directly solving the normal equation. It also allows the algorithm to converge gradually toward the optimal parameters without requiring decomposition methods such as QR factorization (Bottou).

3.2.2. Ridge Regression Complexity

Ridge Regression extends the linear regression model by introducing an L_2 regularization. Which is used to penalize large coefficients, reducing overfitting and improving stability in the presence of multicollinearity (Zhang and Politis; Ting Tin Tin et al.). Ridge Regression uses the same mathematical foundation as the linear regression algorithm, but it adds λ : $(X^T X + \lambda I)\beta = X^T y$, with λ having the role of regularization parameter. The inclusion of this penalty term adds minimal computational overhead, and the total training complexity becomes $O(nm^2 + m^3)$.

3.2.3. Random Forest Complexity

Random Forest Regression achieves strong predictive performance by averaging the outputs of multiple decision trees, which reduces variance and stabilizes predictions (Scornet; Mentch and Zhou). Each individual regression tree is trained by recursively splitting the data to reduce variance, and the computational complexity of building one tree is $O(mn \log n)$, where n is the number of samples and m is the number of features.

In the project, each tree is trained on a bootstrap sample of size $k = \text{sampleRatio} \cdot n$, giving a per-tree cost of $O(km \log k)$. For a forest of t trees, the total training complexity is therefore:

$$O(t \cdot k m \log k)$$

Equation 1. Training Complexity of Random Forest.

Random Forests are naturally parallelizable, since each tree is trained independently. This does not change the theoretical complexity, but it significantly reduces training time, making Random Forests efficient and scalable for medium-sized datasets.

3.2.4. Gradient Boosting Complexity

Gradient Boosting Regression enhances predictive accuracy by building trees sequentially, with each new tree, errors are corrected in an attempt to make a better prediction (Friedman). The training complexity of Gradient Boosting is:

$$O(t \cdot k \cdot \log k)$$

Equation 2. Time Complexity of Gradient Boosting.

Where t represents the number of boosting iterations. While this makes boosting models more computationally demanding compared to Random Forests, they generally achieve higher accuracy when tuned appropriately. Research has demonstrated that boosting techniques, particularly XGBoost (which is a more complex expansion on gradient boosting), outperform traditional models in capturing nonlinear relationships between vehicle attributes and price (Qian; Zheng; Bergmann and Feuerriegel).

3.3. Software Architecture

The application is developed using the Model-View-Controller (MVC) pattern. This structure ensures separation of the responsibilities of data management, user interface, and central logic. The backend, developed with ASP.NET Core, which implements the Model and Controller components from the MVC pattern. And the View part of the pattern is handled by a React and TypeScript in a single-page application (SPA).

In this architecture, the Model is responsible for the machine learning and data processing logic which is responsible for training, evaluating, and serving predictions for used car prices. And it implements the following algorithms - Linear Regression, Ridge Regression, Random Forest, and Gradient Boosting, including the residual-learning combination (Ridge + RandomForest/GradientBoosting) - along with the data

preprocessing pipeline that handles tasks such as feature encoding, scaling, and dataset validation.

The Controller acts as the middleman between the UI and the backend. Its main functionality is to expose RESTful endpoints which establish the communication between the frontend and the backend. Each controller receives input data from the user, invokes the corresponding code and returns structured JSON responses containing what was requested.

The View component is implemented as a React-based web interface. It provides a user-friendly UI where users can and visualize prediction results, compare the performance of different algorithms and compare different car prices throughout the years. The React application communicates asynchronously with the backend through HTTP requests.

Supporting modules are also added in addition to the main MVC components. They provide additional services such as feature scaling, label scaling, data validation, and model serialization. These utility classes maintain the consistency and reusability of the system across different operations. The interactions between all layers of the project are showcased in Figure 4:

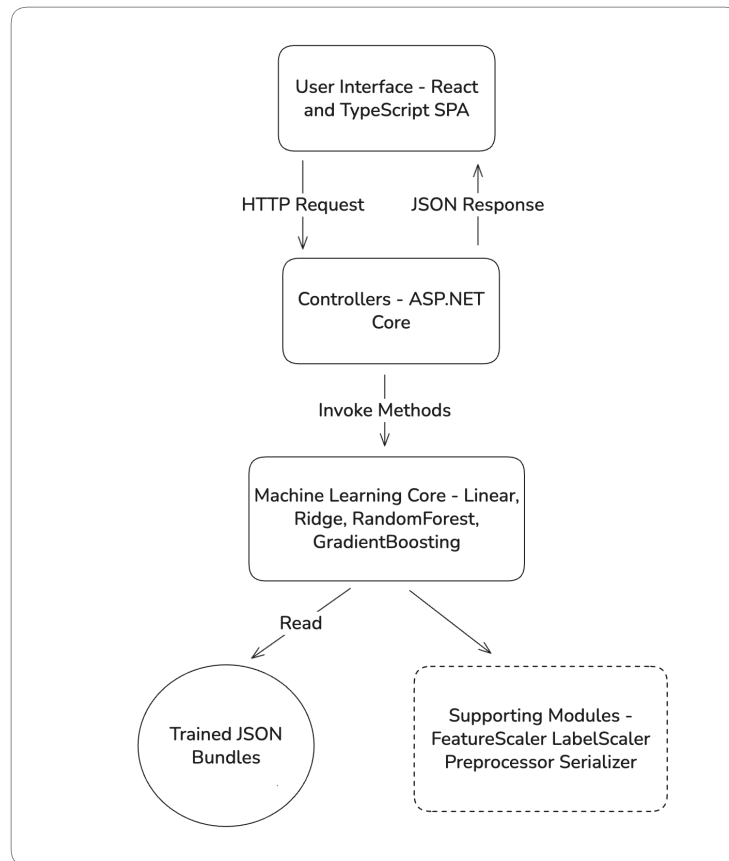


Figure 4. System Achitecture Diagram.

The MVC design pattern was selected because it enables a clear separation between the frontend and the backend, which is essential for a web-based architecture that combines a React frontend and a .NET backend. Another way this can be developed is with monolithic designs but in this situation all UI elements and business logic coexists in the same codebase which makes everything much more couples. On the other hand, the MVC structure provides a way for the data processing and machine learning components remain fully independent from the front end. This split between the two parts of the project is very crucial and it gives the oportunity for easier future development.

3.3.1. Model(s)

There are four main algorithms - Linear Regression, Ridge Regression, Random Forest, and Gradient Boosting. They provide the core predictive functionality of the application. They form the backend and have been implemented according to object-oriented programming (OOP) principles.

At the highest level, an abstract class named *Regressor*. This class defines the common interface and functionality which is shared between all the models. It includes methods for model training and prediction (*Fit()* and *Predict()*). Also there is a set of evaluation utilities for computing key performance metrics: coefficient of determination (R^2), Mean Absolute Error (MAE), and Root Mean Squared Error (RMSE).

The *LinearRegression* class implements the core regression logic and serves as the base implementation for linear modeling. The *RidgeRegression* class inherits from *LinearRegression* and overrides the training logic by adding regularization to the weight updates. The *DecisionTreeRegressor* class serves as a reusable component for *RandomForestRegressor* and *GradientBoostingRegressor*.

The *DecisionTreeRegressor* class serves as a reusable component rather than a standalone predictive model. It defines the logic for recursive tree construction, which is reused by more complex ensemble algorithms. Both the *RandomForestRegressor* and *GradientBoostingRegressor* classes inherit from this component internally. Each of them inherits from *IR regressor* and uses multiple instances of *DecisionTreeRegressor* to accommodate their needs.

The implemented hierarchy ensures high modularity and reusability and scalability. All regression models share a common structure defined by *IR regressor*, while tree-based algorithms extend their functionality by combining multiple decision trees. With this approach there is code duplication and simplifies future expansion, as new regression types can be added by implementing or extending the same base interface.

The high-level class hierarchy, with the relevant inheritance relationships and shared interfaces, is illustrated in Figure 5.

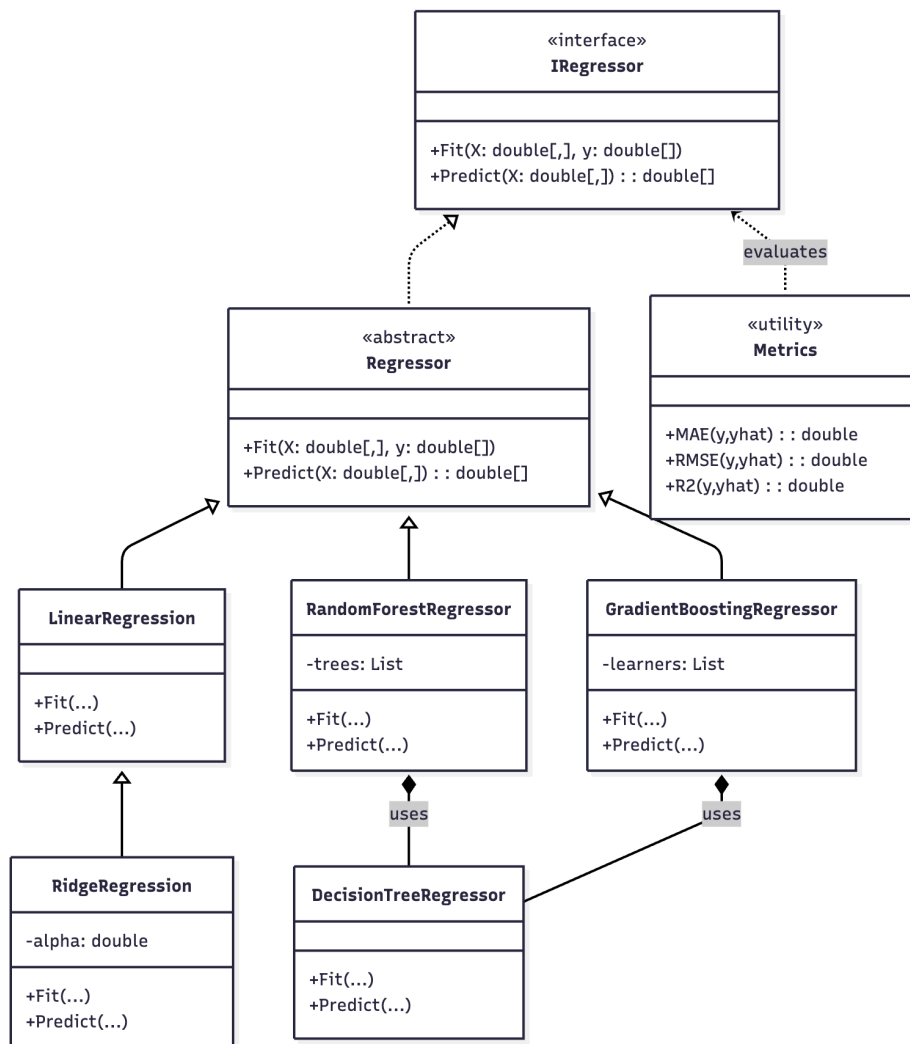


Figure 5. Regression Model Class Hierarchy.

3.3.2. View(s)

The view layer represents the UI of the application and is implemented as a SPA using TypeScript with React. The UI provides an intuitive and minimal way for users to

interact with the prediction system without requiring prior knowledge of machine learning concepts.

The entry point to the SPA is the Main Menu, which serves as the navigation hub for the application. From there, users have the choice to make a prediction in 4 different ways:

- Single Prediction - predicting the price of a single car for a specific year using all available algorithms.
- Single Ranged Prediction - visualizes the predicted price trend of a single car model across a selected year range using all algorithms including a line chart to visualise the trend.
- Compare Models - displays the performance of all algorithms using a bar chart and a table which contains the evaluation metrics (MAE, RMSE, and R^2).
- Compare Ranged Prediction - compares two different car models across a range of years using a selected algorithm, providing line chart visualizations of price evolution.

The UI makes an asynchronous communication with the backend through HTTP requests which include the payload of the car that we want to make a prediction for. The Prediction Controller receives the requests, executes the prediction, and it returns a structured JSON response to the UI. The results show dynamically in the web browser with the use of React States and Charts. Also, each view has a consistent structure with a “Go Back”, “Go Home” and “Next buttons”. This ensures that the UI is smooth throughout all pages.

3.3.2.1. Main Dashboard Overview

The Main Dashboard has 4 options as mentioned in the point above. Its design is kept minimal to ensure simplicity when using the platform. Each prediction view begins with a form where the user provides vehicle-specific attributes.

3.3.2.2. Single Prediction View

The Single Prediction view allows users to obtain a price prediction for a specific car and get predictions based on all implemented algorithms - Linear Regression, Ridge Regression, Random Forest, and Gradient Boosting.

The user has to enter the following inputs to get a prediction:

- Manufacturer
- Model
- Year of Production
- Target Year
- Mileage (Odometer Reading)
- Transmission Type
- Fuel Type

After filling all inputs, the user makes a prediction request by clicking on the “Predict” button. The entered data is sent as a JSON payload to the backend through the Prediction Controller, which returns the results for each algorithm.

The output includes both price predictions and evaluation metrics (MAE, MSE, and R^2), alongside timing information for each model. The results are presented in a structured list that updates dynamically using React states. The view includes

navigation buttons for “Go Back”, “Go Home”, and “Next”, ensuring a smooth transition between different sections of the UI.

3.3.2.3. Single Ranged Prediction View

The Single Ranged Prediction view adds more on top of the single prediction by giving users the option to observe how a car’s price is going to change across a selected year range. This view uses all algorithms to produce multiple predictions for each year within the time range.

The input attributes are the same as in the single prediction view with the only difference being the inclusion of a starting year and an ending year. Once submitted, the backend processes the range by predicting prices for every year within the specified year interval.

The results are visualized using a line chart, where one line shows each algorithms’ predictions for the specified interval. This way users can easily identify what prediction each algorithm has made for the interval of years and also visualise the long-term depreciation of the vehicle. Below the chart, the corresponding evaluation metrics and timing data are displayed as in the single prediction view.

3.3.2.4. Compare Models View

The Compare Models compares the overall performance of all four algorithms for two cars. The inputs are the same as in the single prediction view but the user is required to enter them twice because this view requires two vehicles.

On the results screen the user is presented with a bar chart that contrasts algorithms based on their price predictions. Under the chart, a metrics table displays the numerical results of all metrics and timings of each algorithm.

3.3.2.5. Compare Ranged Prediction View

The Compare Ranged Prediction view enables a direct comparison between two car models over a interval of years. The inputs are the same as in the Ranged Prediction with the small change that the user has to select two different cars and also select a preferred algorithm.

The backend processes both inputs across the same year range and returns the predictions. The results are visualized using a dual-line chart, where each line represents the predictions of the algorithm for one car in the desired interval.

3.3.3. Controller(s)

The Controller(s) connects the user interface and the machine learning logic. The goal of the Controller(s) is to process user requests, execute the models which will produce the prediction. And at the end structure the predictions into a JSON and return it to the front end.

The main controller in this application is the PredictionController. It is responsible for handling all prediction. It is has exposed 4 endpoints under “.../api/v1/prediction”. Which are used accordingly to the view which is selected in the UI.

Each request sent to the PredictionController contains car-specific payload with the following inputs: manufacturer, model, year of production, mileage, transmission, fuel

type, target year, start year(ranged requests only), end year(ranged requests only), algorithm type(ranged requests only). When the controller receives the data it makes a input validation and delegates the computation to the ActiveModel and ModelHotLoader services. They are responsible for loading the correct trained model bundle from the processed dataset folder. After that the appropriate algorithms are called - Linear Regression, Ridge Regression, Random Forest, and Gradient Boosting - to generate predictions and compute the performance metrics.

In the end when the results are computed, the controller structures them into a JSON response that is sent back to the frontend. And they are displayed through charts and tables. The PredictionController does not contain any computational logic by itself. It is more like a orchestrator for the communication between the backend components.

3.4. Primary Vehicle Dataset and Feature Selection

The system is designed to support different datasets but the initial implementation focuses on a single dataset containing used car listings collected from publicly available dataset. The dataset includes vehicles of various brands, models and it is sufficient for model training and evaluation. The data contains the following attributes manufacturer, model, year of production, mileage, fuel type, transmission, and price.

During preprocessing, categorical variables such as fuel type and transmission are encoded. Year and mileage which are numerica attributes are scaled for performance purposes. Missing and inconsistent records are removed, and outliers are filtered out to preserve data quality.

The target variable for prediction for all models is the price of the vehicle. The remaining features features serve as predictors which help the regressors learn about the relationship between the features and the price of the vehicle.

3.5. Custom Dataset Requirements

The initial implementation doesn't have UI for training but the backend fully supports retraining with custom datasets through the command-line interface (CLI). This makes the application very flexible as the user can provide the application their custom CSV and they can use it to train the models and get predictions based on it.

Custom datasets should be provided in CSV format. The file should include the following columns:

price, year, manufacturer, model, odometer, fuel, transmission

11900, 2017, hyundai, elantra, 45000, gas, automatic

The price, year and odometer columns must be numerical. Fuel and transmission columns require plain text (e.g., "manual", "diesel"). Missing, null, or non-numeric entries in the numeric columns are automatically filtered during preprocessing to prevent training errors.

4. Implementation

4.1. Development Environment and Programming Languages

The project was developed using C# and TypeScript. The backend being developed in ASP.NET Core and the frontend with React. The development environment that was chosen was IntelliJ Rider and Visual Studio Code.

C# was selected for the backend because it is a type safe language, it also has a strong OOP foundation. Also prior knowledge for the language of the author was taken into account when making the decision. The .NET ecosystem also offers a rich set of APIs for file management, mathematical operations, and web hosting, which were used to build the application's training and prediction services. And all of these combined made it the right choice when it came to developing this application. ASP.NET Core was used to develop the server-side logic following the Model-View-Controller (MVC) architecture.

The frontend was developed in TypeScript with React. This was the correct language when it came to building a responsive and maintainable single-page application (SPA). Javascript with React is usually a very common practice when it comes to frontend, but Typescript was chosen because of the type safety it provides. Mainly they were used to ensure what exactly should the frontend receive as a response JSON for the backend. React was chosen for its component-based design which gives opportunity for code reuse.

The development environment of choice was JetBrains Rider, which was used for both backend and shared project configuration. Visual Studio was not selected as the .NET environment of development due to it being deprecated in 2022 for MacOS. Rider was chosen for its code analysis, and Git integration. Also Rider is a cross-platform IDE and its especially convenient for the project to be ran on a Linux or Windows machine. For the frontend, Visual Studio Code was used alongside Node.js and npm, offering powerful extensions for React, syntax highlighting, and integrated terminal support.

4.2. External Libraries and Frameworks

In accordance with the senior project guidelines, all core machine learning algorithms - Linear Regression, Ridge Regression, Random Forest, and Gradient Boosting - are

implemented manually without relying on pre-built machine learning libraries. This ensured a deep understanding of the algorithms' mathematical foundations and allowed fine-grained control over their training, evaluation, and optimization processes.

Several other external libraries and frameworks were implemented to help with the overall functionality of the application. More specifically the area around the data handling from the csv, data communication between the frontend and the backend and for the UI.

On the backend, the project uses ASP.NET Core's built-in framework for routing, dependency injection, and RESTful API handling. Additional utility libraries such as System.Text.Json were used for JSON serialization and deserialization of model data. Microsoft.Extensions.FileProviders was used to locate the folder of the datasets and provide the option to define a custom path to a custom CSV. Swashbuckle package was also included to integrate Swagger UI which provides a dynamic interactive documentation and testing for the API endpoints.

On the frontend, several JavaScript libraries and frameworks were integrated. Chart.js was used for rendering dynamic visualizations for line and bar charts, which allows the users to comprehend all the different prediction trends and compare model performance. For styling, Tailwind CSS was implemented for consistent UI development through composable classes. To complement this, the Shadcn/UI component library was implemented to provide elegant, pre-styled React components while maintaining flexibility for customization.

The integration of the frameworks was carefully chosen to maintain project originality while supporting efficiency, scalability, and user experience.

4.3. Installation Requirements and Project Setup

To run the full application locally, the following software must be installed:

- .NET SDK 8.0 or newer - required to build and run the backend (ASP.NET Core).
- Node.js (v18 or newer) and npm - required to build the frontend (React + TypeScript).
- A modern browser such as Chrome, Edge, or Firefox.

Once the prerequisites are installed, follow these steps to build and start the project:

1. Clone the repository

```
1. git clone <repository-url>
2. cd used-car-predictor
```

2. Build the Frontend
Navigate into the ui folder and install all dependencies:
`cd ui`
`npm install`
`npm run build`

This compiles the React + TypeScript interface into a production build located in `ui/dist`, which the backend will automatically serve.

3. Prepare and Run the Backend
Return to the project root and restore NuGet dependencies:

```
1. cd ../
2. dotnet restore
3. dotnet run
```

The API will become available at <https://localhost:5001> (or <http://localhost:5000>) and the compiled frontend can be accessed at the same address.

4. Command-Line Training Utility

The backend includes a CLI tool that allows retraining models on a custom dataset.

```
1. dotnet run
2.     -- train --dataset "path/to/your_dataset.csv"
3.     --manufacturer "Toyota" --model "Yaris"
4.     --anchor-year 2024 --max-configs 10 --min-r2 0.5
```

Common arguments:

- `--dataset <path>` - path to a CSV file containing the required columns (price, year, manufacturer, model, odometer, fuel, transmission). If not provided it will look for the csv in the `datasets/raw` folder.
- `--manufacturer <name>` - Specifies a manufacturer to train only models for that brand.

Can also be used as `--make <name>` (alias).

- `--model <name>` - Trains a specific model of the given manufacturer.

When used, `--manufacturer` must also be provided.

- `--anchor-year <year>` - sets the reference year for training. If not provided it will default to 2030.
- `--max-configs <n>` - limits the number of hyperparameter configurations to evaluate.
- `--min-r2 <value>` - Minimum acceptable R^2 value for all trained algorithms. Defaults to 0.5. Models below this threshold are skipped.
- `--max-mae <value>` - Maximum acceptable Mean Absolute Error. If exceeded, the model is skipped.

- `--max-rmse <value>` - Maximum acceptable Root Mean Squared Error. If exceeded, the model is skipped.

The CLI displays model metrics (MAE, RMSE, R^2) and stores trained bundles under `Backend/datasets/processed`.

These steps ensure that any user with .NET and Node.js installed can clone, build, and run both layers of the project without additional configuration.

4.4. Code Fragments

This section presents key implementation details of the regression algorithms which have been developed in the project. The application supports four algorithms - Linear Regression, Ridge Regression, Random Forest, and Gradient Boosting. I am going to make subsections to focus on the two most central models which are : Ridge Regression, and the ensemble-based methods that rely on reusable `DecisionTreeRegressor` logic.

4.4.1. Ridge Regression

Ridge Regression extends the implementation of linear regression by adding an L_2 regularization term. And it serves as the primary version of a linear model. By adding this penalty large coefficient values are penalized which results in a more stable model when it comes to noisy features and presence of multicollinearity.

The Ridge Regression implementation in the project supports both gradient descent and closed-form solutions. During training, the model minimizes the following objective function:

$$J(w) = \left(\frac{1}{n}\right) \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^p w_j^2$$

Equation 3. Ridge Regression Objective Function.

Where λ (lambda) is the regularization parameter controlling the trade-off between bias and variance.

```
1. function TrainRidge(X, y, alpha):
2.     XTX = Transpose(X) * X
3.     Regularization = alpha * IdentityMatrix
4.     Weights = Inverse(XTX + Regularization) * Transpose(X) * y
5.     return Weights
```

K-fold cross-validation procedure is also implemented which tests multiple α values. Trying to achieve the optimal regularization strength.

4.4.2. DecisionTreeRegressor (Base Component)

The DecisionTreeRegressor class is not used to make predictions by itself. It acts as a reusable component. It encapsulates the recursive logic for splitting the dataset into smaller subsets based on variance reduction which is reused for both the RandomForestRegressor and GradientBoostingRegressor internally. They both use this implementation to build and train multiple trees efficiently and without code duplication.

During training, the algorithm searches for the best feature and threshold that minimize prediction error on each split. The process is executed recursively until the maximum depth or minimum samples per leaf are reached. The leaf node stores the average target value of the subset, which serves as the prediction for unseen data.

```

1. function BuildTree(X, y, depth):
2.     if depth >= maxDepth or len(X) < minSamplesSplit:
3.         return Leaf(mean(y))
4.
5.     bestSplit = FindBestSplit(X, y)
6.     if bestSplit is None:
7.         return Leaf(mean(y))
8.
9.     (leftX, leftY, rightX, rightY) = Partition(X, y, bestSplit)
10.
11.     leftNode = BuildTree(leftX, leftY, depth + 1)
12.     rightNode = BuildTree(rightX, rightY, depth + 1)
13.
14.     return Node(bestSplit.feature, bestSplit.threshold, leftNode, rightNode)

```

4.4.3. Ensemble Algorithms: Random Forest and Gradient Boosting

Both Random Forest and Gradient Boosting regressors extend the reusable DecisionTreeRegressor logic but differ in how they combine trees:

- Random Forest builds multiple trees independently on random subsets of the dataset. The final prediction is the average of all tree outputs.

```

1. function TrainRandomForest(X, y, numTrees):
2.     forest = []
3.     for i in 1..numTrees:
4.         (sampleX, sampleY) = BootstrapSample(X, y)
5.         tree = BuildTree(sampleX, sampleY)
6.         forest.append(tree)
7.     return forest
8.
9. function PredictRandomForest(forest, x):
10.    return Mean([PredictTree(tree, x) for tree in forest])

```

- Gradient Boosting, in contrast, builds trees sequentially. Each new tree learns from the residual errors of the previous ensemble, iteratively reducing prediction error.

```
1. function TrainGradientBoosting(X, y, numTrees, learningRate):
2.     predictions = [0] * len(y)
3.     for i in 1..numTrees:
4.         residuals = y - predictions
5.         tree = BuildTree(X, residuals)
6.         predictions += learningRate * PredictTree(tree, X)
7.     return predictions
```

5. Testing

In this project I have used standard regression metrics - Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and Coefficient of Determination (R^2). To ensure that the project can achieve maximum accuracy and computational efficiency of each of the algorithms. These tests ensure that the predictiveness of the project performs as expected in real-world conditions. And it also verifies that the models implemented in the project deliver reliable and interpretable results.

The evaluations which are focused on determining how each model performs predictions are based on key attributes such as year of production, mileage, transmission type, and fuel type.

Additionally, a training time of each algorithm was also analysed during training time to assess computational efficiency and scalability. These timing metrics provide further information to what are the trade-offs between model complexity and processing cost. This information is very important in the case where a user wants to train dataset of models and needs to do it fast and reliable at the same time.

During testing, an issue was identified with the Random Forest and Gradient Boosting models, where predictions occasionally returned unrealistic values when trying to predict values which were not on par with the year the models were trained on. The problem was due to inconsistencies between the scaling pipeline and model training procedure. The issue was resolved by implementing a residual-learning approach. Ridge Regression combines with ensemble learners like the Random Forest and Gradient Boosting so that it can stabilize predictions and improve accuracy throughout the years.

The results of all tests, including accuracy metrics and timing data, were recorded and visualized in tabular in the end of the prediction report.

5.1 Evaluation Metrics and Procedures

The prediction accuracy of regression models is evaluated using four primary quantitative variables, which include Coefficient of Determination (R^2), Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and Training Time.

The reasons why these metrics were selected are that they represent various aspects of model performance: accuracy, sensitivity of errors, stability, and computational efficiency.

5.1.1. Coefficient of Determination (R^2)

One of the most popular statistical measures in assessing the regression models is the Coefficient of Determination (R^2). It measures the extent to which the predicted values are close to the observed values, which is actually a measure of the percentage

of the variance in the dependent variable that is explained by the independent variables.

Mathematically, the total change in the observed data may be represented as the difference between the true values y_i and the mean $\bar{y}$, and the unexplained variation is represented by the difference between the true values and predicted value $\hat{y}_i$.

$$R^2 = 1 - \frac{\left(\sum_{i=1}^n (y_i - \{\widehat{y}\}_i)^2\right)}{\left(\sum_{i=1}^n (y_i - \overline{y})^2\right)}$$

Equation 4. Formula for the Coefficient of Determination.

A value of R^2 near 1 means that the model accounts for the majority of the variance and gives a better fit to the data whereas lower values indicate less predictive power. When predicting car prices, high R^2 values would indicate that the model has been able to capture the relationship between price and other attributes like year of production, mileage or fuel consumption.

5.1.2. Root Mean Squared Error (RMSE)

Root Mean Squared Error (RMSE) is a square root of the mean squared errors between the estimated and the real numbers. It punishes bigger deviations more severely, and is therefore particularly helpful in identifying outliers or high-variance predictions.

$$RMSE = \sqrt{\left\{ \left(\frac{1}{n} \right) \sum_{\{i=1\}}^n (y_i - \widehat{y}_i)^2 \right\}}$$

Equation 5. Formula for the Root Mean Squared Error.

The target variable is of the same unit as this metric, and it is therefore easy to interpret. Since it focuses on large errors, RMSE was especially applicable in the identification of cases in which the model has underestimated or overestimated a value, such as the prediction of an unnaturally low price of a new car or an unnaturally high price of an old car.

5.1.3. Mean Absolute Error (MAE)

Mean Absolute Error (MAE) is an indicator of the mean size of errors in a group of predictions, independent of the direction. It gives a simple average of the proximity of predictions to the real values:

$$MAE = \left(\frac{1}{n} \right) \sum_{\{i=1\}}^n |y_i - \widehat{y}_i|$$

Equation 6. Formula for the Mean Absolute Error.

In comparison to RMSE, the MAE does not prioritize the magnitude of errors, and therefore, it is more resistant to outliers. Within the framework of this project, the general predictive stability of various types of cars and different price range was

assessed with the help of MAE, which provided an interpretable result of the overall reliability of the model.

5.1.4. Training Time and Efficiency

Besides accuracy measures, each algorithm was also trained and the training time was recorded. This is the overall time it takes to train the model and the average time per configuration trial. Computational performance can be used to measure how well each algorithm scales up, particularly when working with large data sets or when retraining models on a regular basis.

The findings indicate the trade-off between accuracy and efficiency: simpler models like Linear Regression and Ridge Regression can be trained faster but can be ineffective in a more complex environment, but more complex models like Random Forest and Gradient Boosting can be trained much more, but are more effective in prediction.

5.2. Functional Requirements Testing - Backend

After validating that the user interface operates reliably and resists invalid inputs, the next step was to verify that all backend functional requirements were met.

Both unit and integration testing were used to ensure that the application's core server logic - including data preprocessing, prediction, and model-loading mechanisms - behaved correctly under a wide range of scenarios.

Automated tests were implemented using xUnit and FluentAssertions, covering the API controllers, services, and data serialization layers.

Manual acceptance testing was also performed through the browser and API endpoints to confirm that the predicted values and metrics matched expectations.

5.2.1. Prediction Algorithms and Model Loading - Passed

To ensure the prediction everything functioned correctly, the backend was tested against multiple all trained models. Each algorithm calls, the API endpoint */api/v1/prediction/predict* with correct vehicle parameters (manufacturer, model, year of production, mileage, fuel type, and transmission).

The response was expected to contain a set of valid predictions and associated evaluation metrics (MAE, MSE, R^2).

Integration tests verified that the controller correctly invoked the ActiveModel service, loaded the appropriate bundle via the ModelHotLoader, and returned well-structured JSON responses.

Edge-case tests also confirmed that missing, invalid, or mismatched input data resulted in proper HTTP 400 or 503 responses.

All algorithm and model-loading tests passed successfully, confirming that the application can dynamically switch between trained models and consistently produce valid price predictions.

5.2.2. Data Preprocessing and Encoding - Passed

Accurate predictions depend on correct data preprocessing and encoding of categorical variables such as fuel type and transmission.

Unit tests verified that the `ServingHelpers` and `ActiveModel` classes correctly normalized and de-normalized feature vectors and labels, and that categorical encodings matched the vocabulary of the active model.

Additional integration tests confirmed that requests with unsupported or misspelled values (e.g., “diesell”, “hydraulic”) were properly rejected with 400 responses.

The internal scaling functions were validated against manually computed values to ensure numerical correctness.

All preprocessing tests passed, confirming that the backend accurately transforms user input into the feature format expected by the trained algorithms.

5.2.3. API and Controller Reliability - Passed

A comprehensive suite of controller-level integration tests was developed to cover all REST endpoints:

- `/api/v1/prediction/predict`
- `/api/v1/prediction/predict/range`
- `/api/v1/prediction/predict-two`
- `/api/v1/prediction/predict-two/range`
- `/api/v1/models/list`
- `/api/v1/models/details`
- `/api/v1/manufacturers`
- `/health`

Overall, the backend tests achieved a code coverage of 84%, which indicates that almost all paths, validation rules, and data flows were tested.

5.2.4. Minimum Prediction Accuracy - Passed

Minimum performance accuracy was verified using acceptance testing. The metrics (MAE, MSE, R2) of each trained model were tested to ensure that they had reached the minimum values as stipulated in the requirements. The tests achieved the expected accuracy. They demonstrated better accuracy than expected.

5.3. Functional Requirements Testing - Frontend

The frontend was validated with an suite of unit and integration tests implemented using Jest and React Testing Library. The main goal was to ensure that all UI ways of accessing the project (single prediction, range prediction, two-car comparison) act as expected. Also that form validation and data transformations working as expected. Tests were implemented for visual components (cards and charts) to ensure that they render consistent results when fed with mocked API responses.

5.3.1. UI Components & State Handling - Passed

Core components - including AlgorithmMetricsCard, AlgorithmResultCard, ModelInfoCard, PriceRangeChart, RangeComparisonChart, and ComparisonChart - were tested for rendering, prop handling, and edge-case rendering (e.g., empty states and metric rounding). Hooks were covered to verify loading/error states and transformation logic when rendering mocked responses.

5.3.2. Form Validation & User Flows - Passed

PredictionForm and page-level flows (SinglePrediction, RangePrediction, ComparisonPrediction, RangeComparisonPrediction) were tested to verify input constraints (manufacturer, model, years, mileage, fuel type, transmission), client-side validation feedback. The validation utilities were covered with direct unit tests to ensure consistent formatting.

5.3.3. Data Transformation & View Consistency - Passed

Utilities and page logic were verified to ensure transformations behave as expected. Tests confirm that algorithm metrics and prediction outputs are mapped to the expected visual structures.

5.3.4. Integration with Backend Contracts - Passed

All components that depend on the controller responses components and hooks were tested against mocked responses to validate behaviour without relying on a live server. Tests assert that successful responses hydrate the UI correctly. Also simulated error responses trigger appropriate user feedback.

5.3.5. Coverage Summary

The frontend test suite achieved 96.8% statements, 89.67% branches, 98.86% functions, and 96.62% lines across 16 test suites (213 tests). Per-area highlights include:

- src and src/lib at 100% across all metrics.

- src/components and src/hooks at 100% statements/lines/functions, with high branch coverage (e.g., AlgorithmMetricsCard and PredictionForm cover alternative display paths and validation outcomes).
- Page components (src/pages) maintain >94% statements/lines, with remaining uncovered branches corresponding to rare UI paths and defensive fallbacks.

6. Results and Conclusion

6.1 Experimental Setup

The project has been evaluated on the trained bundles generated by the CLI trainer (anchor year set as indicated on each bundle). All predictions were made through the production API and visualized in the Front-end UI. The minimum acceptable performance was $R^2 \geq 0.50$ and minimum entries for a model are 50; bundles failing this threshold were automatically skipped at training time.

6.2 Algorithm Performance (Single Prediction)

The evaluation results for the single-prediction scenario are presented in *Figure 6*. Both Linear and Ridge Regression produced identical outcomes with an R^2 of 0.75 and a mean absolute error (MAE) of approximately 1 895 \$. This indicated a well-fitted linear relationship between the independent variables and the target price. The Random Forest model delivered a considerable improvement, reaching $R^2 = 0.86$ and $MAE \approx 1\,334$ \$. Gradient Boosting algorithm achieved the best accuracy, with $R^2 = 0.88$ and $MAE \approx 1\,226$ \$. The MSE was reduced from 8.1×10^6 for the linear models to 3.9×10^6 for Gradient Boosting.

Regarding efficiency, Ridge Regression trained the fastest (≈ 0.2 s total). Gradient Boosting required the longest runtime (≈ 410 s). All four models exceeded the minimum performance requirement of $R^2 \geq 0.5$. Which shows that all algorithms are able to predict with very good quality. From this we can confirm that the ensemble-based methods (Gradient Boosting and Random Forest), deliver the highest predictive power for complex used-car price estimation tasks. On the other side, Ridge Regression remains an efficient baseline for rapid evaluation.

Algorithm Performance Metrics			
Linear Regression		Ridge Regression	
Metrics:		Metrics:	
MSE:	8090455.35	MSE:	8104326.59
MAE:	1891.52	MAE:	1895.39
R^2 :	0.750	R^2 :	0.750
Timing:		Timing:	
Total Time:	1.385 s	Total Time:	0.198 s
		Trials:	200
		Mean Trial:	0.001 s
Random Forest		Gradient Boosting	
Metrics:		Metrics:	
MSE:	4473092.67	MSE:	3924617.62
MAE:	1334.63	MAE:	1226.02
R^2 :	0.860	R^2 :	0.880
Timing:		Timing:	
Total Time:	43.774 s	Total Time:	410.008 s
Trials:	60	Trials:	60
Mean Trial:	0.730 s	Mean Trial:	6.833 s

Figure 6. Algorithm performance metrics for single-car prediction.

6.3 Price Prediction Over Time (Range Prediction)

To examine how the trained models behave across future target years, a range prediction experiment was performed for a *Toyota Corolla (2021, Automatic, Gas, 80 000 km)* using a bundle trained with an anchor year of 2030. Figures 7 – 9 present

the quantitative metrics, the model information, and the predicted price trajectory between 2025 and 2030.

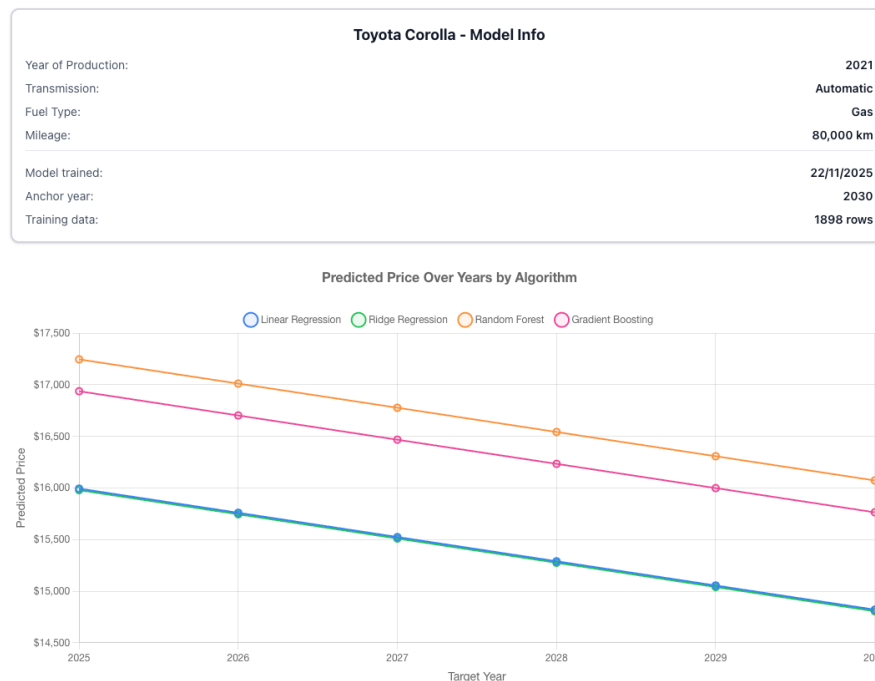


Figure 7. Predicted price trajectory between the years 2025-2030 for Toyota Corolla bundle.

All four algorithms showcase a consistent, monotonic decline in predicted price, which accurately represents the expected depreciation of the vehicle over time. Linear and Ridge Regression models produced nearly identical results, starting from roughly \$15 980 in 2025 and dropping to $\approx$ \$14 800 by 2030. Both ensembles - Random Forest and Gradient Boosting - predicted slightly higher absolute values across all years. Random forest near \$17 200 and Gradient Boosing around \$16 940 in 2025 respectively. They use Ridge Regression as their base estimator and they maintained the same overall downward trend but captured finer nonlinear variations in the rate of depreciation. The rate of depreciation captured by all models was gradual and stable, confirming that the anchor-year training strategy yields models that generalise sensibly when extrapolating beyond the year of training.

Algorithm Performance Metrics			
Linear Regression		Ridge Regression	
Metrics:		Metrics:	
MSE:	8090455.35	MSE:	8104326.59
MAE:	1891.52	MAE:	1895.39
R ² :	0.750	R ² :	0.750
Timing:		Timing:	
Total Time:	1.385 s	Total Time:	0.198 s
		Trials:	200
		Mean Trial:	0.001 s
Random Forest		Gradient Boosting	
Metrics:		Metrics:	
MSE:	4473092.67	MSE:	3924617.62
MAE:	1334.63	MAE:	1226.02
R ² :	0.860	R ² :	0.880
Timing:		Timing:	
Total Time:	43.774 s	Total Time:	410.008 s
Trials:	60	Trials:	60
Mean Trial:	0.730 s	Mean Trial:	6.833 s

Figure 8. Algorithm performance metrics for range car prediction.

Quantitatively, Gradient Boosting once again achieved the strongest performance ($R^2 = 0.88$, $MAE \approx 1\,226$ \$), followed by Random Forest ($R^2 = 0.86$, $MAE \approx 1\,335$ \$). Linear and Ridge Regression reached $R^2 = 0.75$ with $MAE \approx 1\,890$ \$, remaining well above the project's minimum accuracy target of $R^2 \geq 0.5$. These results indicate that while simpler models capture the dominant linear depreciation trend, ensemble methods better approximate subtle nonlinear effects such as interaction between mileage and production year. Overall, the price-over-time analysis confirms that the models remain stable and economically interpretable across multi-year forecasts.

Year 2025			
Linear Regression Predicted Price \$15,994	Ridge Regression Predicted Price \$15,979	Random Forest Predicted Price \$17,247	Gradient Boosting Predicted Price \$16,938
Year 2026			
Linear Regression Predicted Price \$15,760	Ridge Regression Predicted Price \$15,745	Random Forest Predicted Price \$17,012	Gradient Boosting Predicted Price \$16,703
Year 2027			
Linear Regression Predicted Price \$15,525	Ridge Regression Predicted Price \$15,510	Random Forest Predicted Price \$16,778	Gradient Boosting Predicted Price \$16,468
Year 2028			
Linear Regression Predicted Price \$15,290	Ridge Regression Predicted Price \$15,275	Random Forest Predicted Price \$16,543	Gradient Boosting Predicted Price \$16,234
Year 2029			
Linear Regression Predicted Price \$15,055	Ridge Regression Predicted Price \$15,040	Random Forest Predicted Price \$16,308	Gradient Boosting Predicted Price \$15,999
Year 2030			
Linear Regression Predicted Price \$14,821	Ridge Regression Predicted Price \$14,805	Random Forest Predicted Price \$16,073	Gradient Boosting Predicted Price \$15,764

Figure 9. Predicted prices by year for all algorithms, showing consistent depreciation trend.

6.4 Comparison Views

The comparison interfaces extend the same predictive logic to multiple vehicles simultaneously. The *Comparison Prediction* view evaluates two cars for a fixed target year, while the *Range Comparison Prediction* view projects both vehicles across a defined year range. Both rely on identical trained bundles and algorithm configurations discussed in Sections 6.2 and 6.3. The results consistently reflect each vehicle's features while depreciation remain parallel across algorithms.

6.5 Future Developments

The project has been successfully completed, meeting all of its functional and non-functional requirements. The project performs reliably and achieves high metrics for all algorithms. It also provides user-friendly visualization of predictions. There are several valuable enhancements that could be implemented to further improve the project. But they do not overshadow what is achieved in this project. They are additional possible directions in which the project can expand on:

1. Training Directly from the Web Interface

Currently model training is initiated through the command-line interface. Allowing users to upload datasets and trigger the training process directly from the web application would make the system far more accessible to non-technical users.

2. Cross-Validation of Models

Currently, training and testing are performed with a fixed split based on the available data. Implementing configurable cross-validation would help ensure that the algorithms generalize well to unseen samples and reduce the risk of overfitting.

3. Feature-Importance and Model-Interpretability Tools

Future versions could improve transparency by visualizing feature importance and showing how individual variables-such as mileage, production year, or fuel type-affect the predicted price. Displaying regression coefficients, feature weights, or decision-tree structures would allow for users insights into the model's behaviour.

4. Support for Additional Algorithms

The current implementation focuses on four regression algorithms: Linear, Ridge, Random Forest, and Gradient Boosting. Extending the framework to

include other approaches - such as XGBoost or even neural networks - could improve performance on larger or more complex datasets.

6.6 Conclusion

This project implements a machine-learning system for predicting used-car prices based on historical market data. It meets all functional and non-functional requirements. It provides a fully operational backend and a responsive web interface for visualising predictions, performance metrics, and model information.

The implemented algorithms - Linear Regression, Ridge Regression, Ridge + Random Forest, and Ridge + Gradient Boosting - demonstrate amazing predictive accuracy. Each model exceeds the minimum performance target of $R^2 \geq 0.5$.

Gradient Boosting and Random Forest achieved the highest precision. Which confirms that their combination with residual learning based on Ridge Regression effectively captures nonlinear relationships between different vehicle features.

In addition to performance of each algorithm, the project focused quality of the project itself and to ensure that the code is well tested. The backend reached 84 % code coverage. The frontend achieved over 96 %. The API's validation and error-handling mechanisms guarantee that all requests valid or invalid will be handled. The frontend presents complex model metrics using user-friendly elements.

Overall, the project proves that machine learning can be effectively applied to used-car price prediction. And it can be achieved by combining reliable regression models with a clear and responsive web user interface. The project provides accurate and easy-to-use forecasts. It forms a solid base for future extensions such as training directly from the web application and algorithm expansion.

7. References

- Alnajim, Thuraya, et al. "An Intelligent Vehicle Price Estimation Approach Using a Deep Neural Network Model." *World Electric Vehicle Journal*, vol. 15, no. 8, 2024, <https://doi.org/10.3390/wevj15080345>.
- Amik, Fahad R., et al. "Application of Machine Learning Techniques to Predict the Price of Pre-Owned Cars in Bangladesh." *Information*, vol. 12, no. 12, 2021, <https://doi.org/10.3390/info12120514>.
- Bergmann, Svenja, and Stefan Feuerriegel. "Machine Learning for Predicting Used Car Resale Prices Using Granular Vehicle Equipment Information." *Expert Systems with Applications*, vol. 263, Mar. 2025, p. 125640, <https://doi.org/10.1016/j.eswa.2024.125640>.
- Bottou, Léon. "Large-Scale Machine Learning with Stochastic Gradient Descent." *Proceedings of COMPSTAT'2010*, edited by Yves Lechevallier and Gilbert Saporta, Physica-Verlag HD, 2010, pp. 177–86.
- Friedman, Jerome H. "Greedy Function Approximation: A Gradient Boosting Machine." *Annals of Statistics*, vol. 29, no. 5, 2001, pp. 1189–232, <https://doi.org/10.1214/aos/1013203451>.

- Gegic, Enis, et al. "Car Price Prediction Using Machine Learning Techniques." *TEM Journal*, vol. 8, no. 1, Feb. 2019, Academic Search Ultimate, 135048125, pp. 113–18. *EBSCOhost*, <https://doi.org/10.18421/TEM81-16>.
- Haque, Alimul, et al. "Implication of Different Data Split Ratio on the Performance of Model in Price Prediction of Used Vehicles Using Regression Analysis." *Data & Metadata*, vol. 3, Jan. 2024, Academic Search Ultimate, 181529805, pp. 1–14. *EBSCOhost*, <https://doi.org/10.56294/dm2024425>.
- J. I. Kang, et al. "Comparing Regression Models Predicting the Price of Used Cars in Big Data." *2022 IEEE International Conference on Consumer Electronics-Asia (ICCE-Asia)*, 2022, pp. 01–04, <https://doi.org/10.1109/ICCE-Asia57006.2022.9954633>.
- Mentch, Lucas, and Siyu Zhou. "Randomization as Regularization: A Degrees of Freedom Explanation for Random Forest Success." *Journal of Machine Learning Research*, vol. 21, no. 171, 2020, pp. 1–36.
- Molnar, Christoph. *Interpretable Machine Learning: A Guide for Making Black Box Models Explainable*. Edited by 2, Christoph Molnar, 2023, <https://christophm.github.io/interpretable-ml-book/feature-importance.html>. Feature Importance.
- Pal, Nabarun, et al. *How Much Is My Car Worth? A Methodology for Predicting Used Cars Prices Using Random Forest*. arXiv preprint. 2017, <https://arxiv.org/abs/1711.06970>.
- Qian, Tingyu. "Used Car Price Prediction by Using XGBoost." *BCP Business & Management*, vol. 44, Apr. 2023, pp. 62–68, <https://doi.org/10.54691/bcpbm.v44i.4794>.

- Salim, Fadzilah, and Nur Abu. "Used Car Price Estimation: Moving from Linear Regression towards a New S-Curve Model." *International Journal of Business and Society*, vol. 22, Dec. 2021, pp. 1174–87, <https://doi.org/10.33736/ijbs.4293.2021>.
- Scornet, Erwan. *Trees, Forests, and Impurity-Based Variable Importance*. 2021, <https://arxiv.org/abs/2001.04295>.
- Ting Tin Tin, et al. "Car Depreciation Price Prediction Using Multiple Machine Learning Algorithms." *Pakistan Journal of Life & Social Sciences*, vol. 22, no. 2, July 2024, Academic Search Ultimate, 185141745, pp. 19239–51. *EBSCOhost*, <https://doi.org/10.57239/PJLSS-2024-22.2.001411>.
- Zhang, Yunyi, and Dimitris N. Politis. *Ridge Regression Revisited: Debiasing, Thresholding and Bootstrap*. 2021, <https://arxiv.org/abs/2009.08071>.
- Zheng, Yufan. "Machine Learning Optimization and Challenges in Used Car Price Prediction." *ITM Web of Conferences*, vol. 70, Jan. 2025, <https://doi.org/10.1051/itmconf/20257004032>.